

Universidad Tecnológica Nacional (UTN)

Tecnicatura Universitaria en Programación

Programación I

**Sistema de gestión y análisis
estadístico de países**

Etura Sofía

Valdez Luciana

15 de Junio de 2026

Índice

Marco teórico	Pág. 3
Estructuras de datos dinámicas (Listas y diccionarios)	Pág. 3
Persistencia de Datos (Archivos CSV)	Pág. 4
Algoritmos de Ordenamiento (Bubble Sort).....	Pág. 4
Manejo de excepciones.....	Pág. 5
Decisiones técnicas y Arquitectura	Pág. 5
Modularización mediante funciones.....	Pág. 5
Control de errores de entrada (Estructura de menú).....	Pág. 6
El motor de búsqueda flexible (Limpieza de cadenas)	Pág. 7
Lógica del Algoritmo de Ordenamiento.	Pág. 7
Capturas de Pantalla de la Ejecución.....	Pág. 8
Dificultades y conclusiones.....	Pág. 16
Dificultades técnicas encontradas y soluciones.	Pág. 16
Conclusiones y aprendizajes	Pág. 17
Bibliografía / Webgrafía.....	Pág. 18
Links.....	Pág. 18

1. Marco teórico

Para entender cómo funciona este programa, podemos imaginarlo como el sistema de organización de un fichero físico en una oficina.

El código se encarga de tres tareas básicas: guardar los datos en la memoria mientras la computadora está encendida, pasarlos a un archivo para no perderlos al apagarla, y ordenarlos o filtrarlos cuando el usuario lo solicita desde el menú.

Estructuras de datos dinámicas (Listas y diccionarios).

Para manejar toda la información en la memoria de la computadora mientras el programa se está ejecutando, combinamos dos herramientas clave de Python: una lista general y muchos diccionarios adentro.

- **La lista (*lista_paises*):** Funciona como nuestro mueble fichero principal. Es una estructura dinámica porque su tamaño no es fijo. Al iniciar el programa está vacía, pero a medida que leemos el archivo o creamos un nuevo registro, usamos el comando `.append()` para sumarlo al final. El mueble se agranda o se achica solo según la cantidad de países que tengamos.
- **El diccionario (*pais*):** Dentro de ese gran mueble, cada país es una "ficha" individual. En vez de guardar los datos sueltos o depender de números de posición (índices), el diccionario nos permite usar etiquetas de texto muy claras llamadas *claves* ('nombre', 'poblacion', 'superficie', 'continente'). Esto hace que nuestro código sea muy fácil de leer. Si el programa quiere imprimir

la población de un país, simplemente busca la etiqueta `p['poblacion']` sin riesgo de confundirse.

Persistencia de datos (Archivos CSV)

La memoria RAM de la computadora pierde toda la información en cuanto el programa se cierra o la máquina se apaga. Para lograr la persistencia (es decir, que los datos se guarden para siempre y se puedan recuperar mañana), el sistema se conecta con un archivo llamado: *países.csv*.

Un archivo CSV, un formato de texto simple que guarda los datos en filas y columnas, que se pueden abrir directamente en Excel.

Para leerlo desde Python usamos la librería nativa CSV con la herramienta DictReader. Lo que hace es leer la primera línea del archivo como si fuesen los títulos de las columnas y, automáticamente, transforma cada renglón de texto en un diccionario con sus etiquetas listas para meter dentro de nuestra lista principal.

Algoritmos de ordenamiento (Bubble Sort)

Cuando el usuario selecciona la opción para ver los países ordenados por población, superficie o nombre, los datos no se acomodan solos. En lugar de usar funciones automáticas de Python, decidimos programar a mano el algoritmo clásico Bubble Sort para aplicar la lógica de la materia.

El algoritmo funciona mediante dos bucles for anidados que recorren la lista y comparan los elementos de dos en dos. Si el programa detecta que el primer país es

mayor o menor que el segundo (según el sentido que elija el usuario), realiza un intercambio de lugares en la memoria.

Se llama "Burbuja" porque los valores más grandes van "viajando" poco a poco hacia el final de la colección, tal como suben las burbujas de una gaseosa hacia la superficie.

Manejo de Excepciones

Un programa tiene que ser seguro y no puede romperse ni cerrarse solo si el usuario comete un error al tipear. En las opciones de agregar o actualizar un país, si el sistema pide la población (un número entero) y el usuario escribe letras por error, un programa común de Python fallaría críticamente arrojando un error de sistema (ValueError).

Para evitar esto, usamos los bloques de control try-except (intentar / capturar error) combinados con bucles while True. Esto funciona como una red de seguridad: el programa intenta convertir la entrada en un número entero. Si el usuario ingresa texto, el bloque except frena ese error antes de que rompa el programa, muestra un mensaje amigable en la consola ("*Dato inválido. Intente nuevamente*") y reinicia el bucle para darle otra oportunidad al operador.

2. Decisiones técnicas y Arquitectura

Modularización mediante funciones

Para que el código sea ordenado, fácil de leer, y no tener un solo bloque gigante de texto, la principal decisión fue dividir el problema en partes.

Aplicamos la modularización: creamos una función específica para cada opción del menú.

Al inicio del programa (en la función *main()*), creamos una lista vacía llamada *lista_paises = []*. Esta lista funciona como nuestra base de datos central en memoria y se la pasamos como argumento a las demás funciones (por ejemplo, *agregar_pais(lista_paises)* o *ordenar_paises(lista_paises)*).

De esta manera, todas las funciones trabajan sobre los mismos datos de forma limpia y coordinada.

Control de errores de entrada (Estructura de menú)

Una decisión de diseño clave para que el programa sea amigable y no se rompa fue el uso de bucles infinitos *while True* combinados con bloques *try-except* dentro de las funciones de carga (*agregar_pais* y *actualizar_pais*).

En lugar de dejar que el usuario ingrese cualquier cosa, el programa se planta en un bucle y exige un dato válido. Por ejemplo, al pedir la población o la superficie:

1. El programa entra en el *while True* y pide el dato.
2. Si el usuario escribe letras, el *except ValueError* frena el error, avisa que el dato es inválido y el *while* vuelve a empezar (no lo deja avanzar).
3. Si el usuario ingresa un número, el código *Medicare* que sea mayor a cero (valor ≤ 0). Si cumple, recién ahí se rompe el bucle con un *break* y pasa al siguiente casillero.

Además, en todas las pantallas de carga le dimos la opción al usuario de arrepentirse ingresando la letra 'x', lo que corta la función inmediatamente con un return y lo devuelve a salvo al menú principal.

El motor de búsqueda flexible (Limpieza de cadenas)

Python es muy estricto al comparar textos: distingue mayúsculas de minúsculas y no entiende que la "á" con tilde es la misma letra que la "a" común. Si el usuario buscaba "ARGENTINA" o "argentina" (sin tilde), el programa original no lo encontraba porque en el archivo CSV estaba guardado como "Argentina".

Para resolver esto de forma inteligente, diseñamos un mecanismo de normalización al vuelo. Cada vez que el usuario busca un país (Opción 4) o quiere modificarlo (Opción 3), agarramos el texto ingresado y el nombre guardado en la lista, y los "limpamos" convirtiéndolos por completo a minúsculas (.lower()) y quitando las tildes a mano con .replace():

```
busqueda_limpia = pais_buscar.lower().replace("á", "a").replace("é", "e").replace("í", "i").replace("ó", "o").replace("ú", "u")
```

Gracias a esta decisión técnica, el buscador se volvió flexible y encuentra los países sin importar cómo los escriba el operador. Además, para la función buscar_pais, usamos el operador in de Python, lo que permite hacer búsquedas parciales (si el usuario escribe "Ar", el programa le va a devolver una tablita con Argentina, Armenia, Aruba, etc.).

Lógica del algoritmo de ordenamiento.

Para la función `ordenar_paises` (Opción 6), decidimos implementar el algoritmo Bubble Sort (Ordenamiento por Burbuja). A través de un diccionario que llamamos `mapa_claves`, el programa traduce la opción del usuario ("1", "2" o "3") al nombre de la etiqueta real en el código ("nombre", "poblacion" o "superficie").

El algoritmo usa dos bucles `for` para comparar elementos vecinos. Si están desordenados, los intercambia de lugar en una sola línea. Como detalle técnico, si el usuario elige ordenar por "nombre", el programa aplica un `.lower()` temporal para que el orden alfabético sea correcto y las mayúsculas no alteren el resultado.

Capturas de pantalla de la ejecución

A. Carga inicial de datos desde el archivo CSV

A continuación se muestra cómo el programa lee exitosamente el archivo `paises.csv` mediante la Opción 1 y despliega los registros formateados en una tabla limpia:

```
GESTIÓN DE DATOS DE PAÍSES

1) Carga inicial de países
2) Agregar nuevo país
3) Actualizar datos de un país
4) Buscar país
5) Filtrar países
6) Ordenar países
7) Estadísticas avanzadas y Salir

Ingrese una opción: 1

-----
NOMBRE DEL PAÍS | POBLACIÓN | SUPERFICIE KM² | CONTINENTE
-----
Argentina      | 46,000,000 | 2,780,400      | America
Canada         | 38,000,000 | 9,984,670      | America
Mexico         | 128,000,000 | 1,964,375      | America
Brasil         | 214,000,000 | 8,515,767      | America
Italia         | 59,000,000 | 301,340        | Europa
España         | 47,000,000 | 505,990        | Europa
Egipto         | 110,000,000 | 1,002,450      | Africa
Japon          | 125,000,000 | 377,975        | Asia
Tailandia       | 71,000,000 | 513,120        | Asia
China          | 1,412,000,000 | 9,596,960     | Asia
Alemania       | 84,000,000 | 357,022        | Europa
-----
```


B. Validación de datos ante ingresos incorrectos

En esta captura se observa el funcionamiento de la Opción 2. El sistema rechaza datos vacíos o textos donde corresponden números (población), demostrando la robustez de los bucles de control:

```
GESTIÓN DE DATOS DE PAÍSES

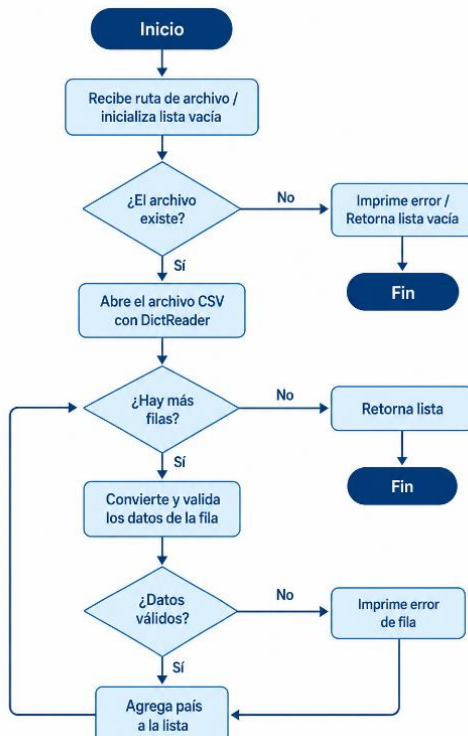
1) Carga inicial de países
2) Agregar nuevo país
3) Actualizar datos de un país
4) Buscar país
5) Filtrar países
6) Ordenar países
7) Estadísticas avanzadas y Salir

Ingrese una opción: 2

AGREGAR NUEVO PAÍS
Ingrese 'x' para cancelar.

Nombre del país:
El nombre no puede estar vacío. Intente nuevamente.
Nombre del país: 
```

Diagramas de flujo.

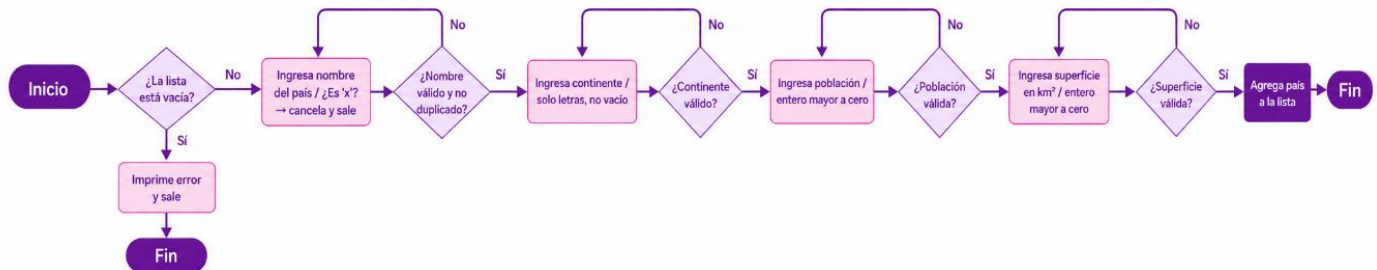


Cargar países del csv:

Este primer diagrama representa La función cargar_paises_csv y es el punto de entrada del sistema, se encarga de leer el archivo csv desde el disco y convertir cada fila en un diccionario Python que representa un país.

Podemos verlo en ejecución en la imagen.

Agregar país:



El diagrama de 'agregar_paises' muestra el flujo que recorre la función para incorporar un nuevo país al sistema con todos los datos requeridos.

A continuación podemos verlo en ejecución:

```

Ingrese una opción: 2

AGREGAR NUEVO PAÍS
Ingrese 'x' para cancelar.

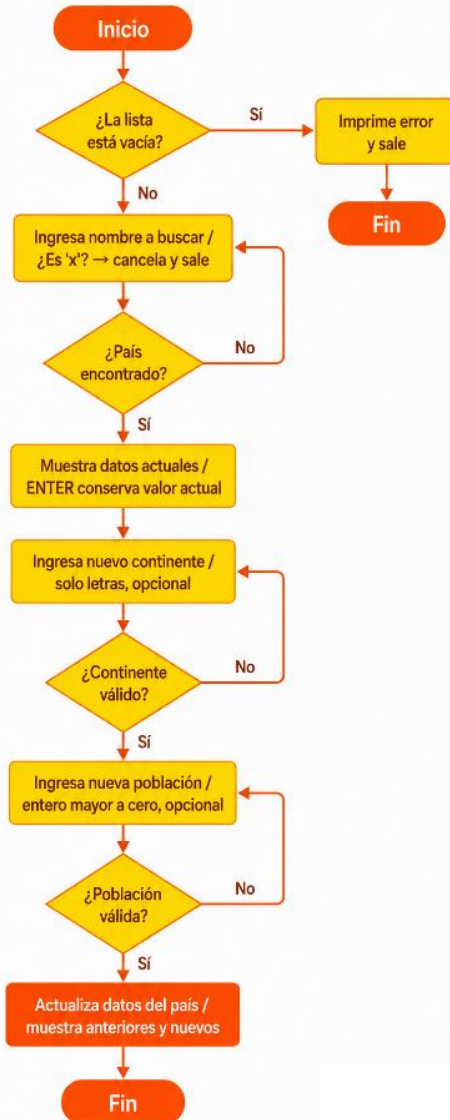
Nombre del país: Corea Del Norte
Continente de Corea Del Norte: Asia
Población de Corea Del Norte: 26633691
Superficie en km² de Corea Del Norte: 120540

'Corea Del Norte' fue agregado correctamente.

-----
NOMBRE DEL PAÍS | POBLACIÓN | SUPERFICIE KM² | CONTINENTE
-----
Argentina       | 46,000,000 | 2,780,400      | America
Canada          | 38,000,000 | 9,984,670      | America
Mexico          | 128,000,000 | 1,964,375      | America
Brasil          | 214,000,000 | 8,515,767      | America
Italia          | 59,000,000 | 301,340        | Europa
España          | 47,000,000 | 505,990        | Europa
Egipto          | 110,000,000 | 1,002,450      | Africa
Japón           | 125,000,000 | 377,975        | Asia
Tailandia        | 71,000,000 | 513,120        | Asia
China           | 1,412,000,000 | 9,596,960     | Asia
Alemania        | 84,000,000 | 357,022        | Europa
Corea Del Norte | 26,633,691 | 120,540        | Asia
-----

```

Actualizar país:



El tercer diagrama representa la función actualizar_pais, esta función permite modificar los datos de un país que ya existe en el sistema.

En la siguiente imagen podemos verla en ejecución:

```
Ingrese una opción: 3

ACTUALIZAR DATOS DE UN PAÍS
Ingrese 'x' para cancelar y volver al menú principal

Ingrese el nombre del país a modificar: Argentina

País seleccionado: Argentina
(Presione ENTER para conservar el valor actual sin cambios)

Nuevo continente: America Del sur
Nueva población:
Nueva superficie en km²:

Los datos de 'Argentina' fueron actualizados.

-----
DATOS ANTERIORES || Continente: America | Población: 46,000,000 | Superficie: 2,780,400 km²
DATOS NUEVOS     || Continente: America Del sur | Población: 46,000,000 | Superficie: 2,780,400 km²
-----
```

Buscar países:

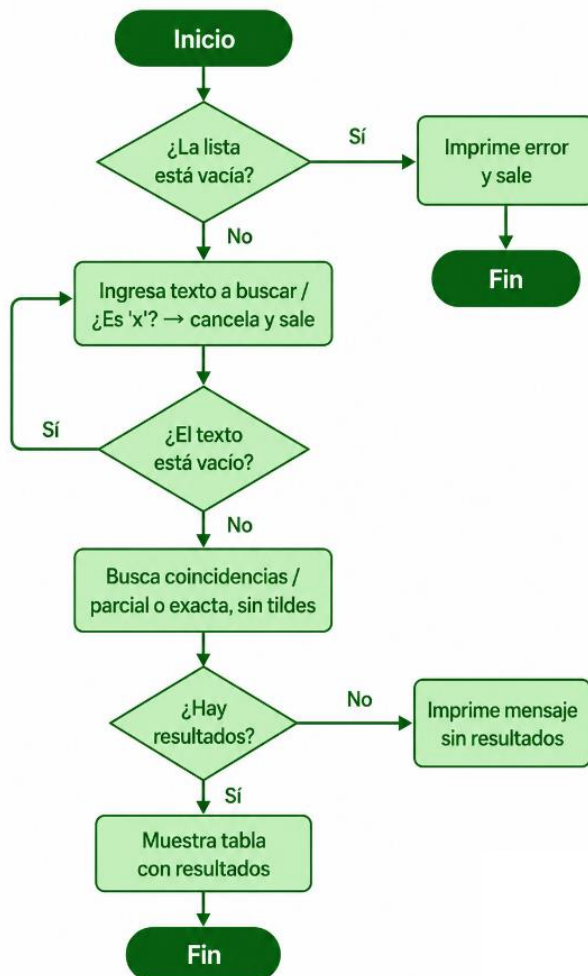


Diagrama representativo para la función `buscar_paises`, esta función permite encontrar uno o más países según el texto ingresado por el usuario. La búsqueda es flexible, es decir que acepta coincidencias parciales o exactas y no distingue tildes, lo que facilita la escritura.

A continuación, podemos verla en ejecución:

```
Ingrese una opción: 4

BUSCAR PAÍS
Ingrese 'x' para cancelar.

Ingrese texto a buscar: arg

Resultados para 'arg':

-----
NOMBRE DEL PAÍS | POBLACIÓN | SUPERFICIE KM² | CONTINENTE
-----
Argentina       | 46,000,000 | 2,780,400      | America Del sur
-----
```

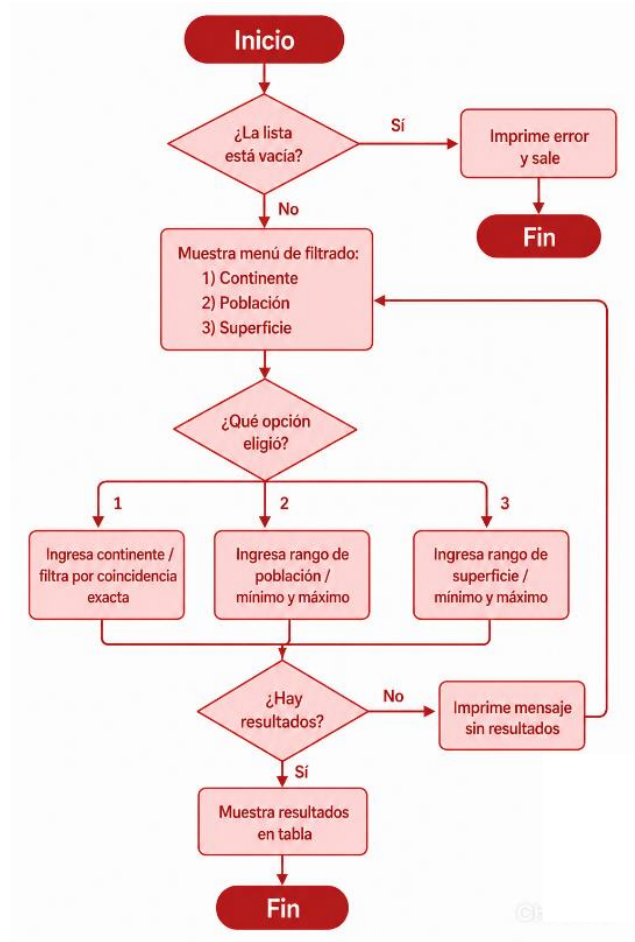
Filtrar países:

Este diagrama representa la función `filtrar_paises` permite acotar la lista de países según tres criterios: continente, rango de población o rango de superficial. A continuación lo vemos en ejecución:

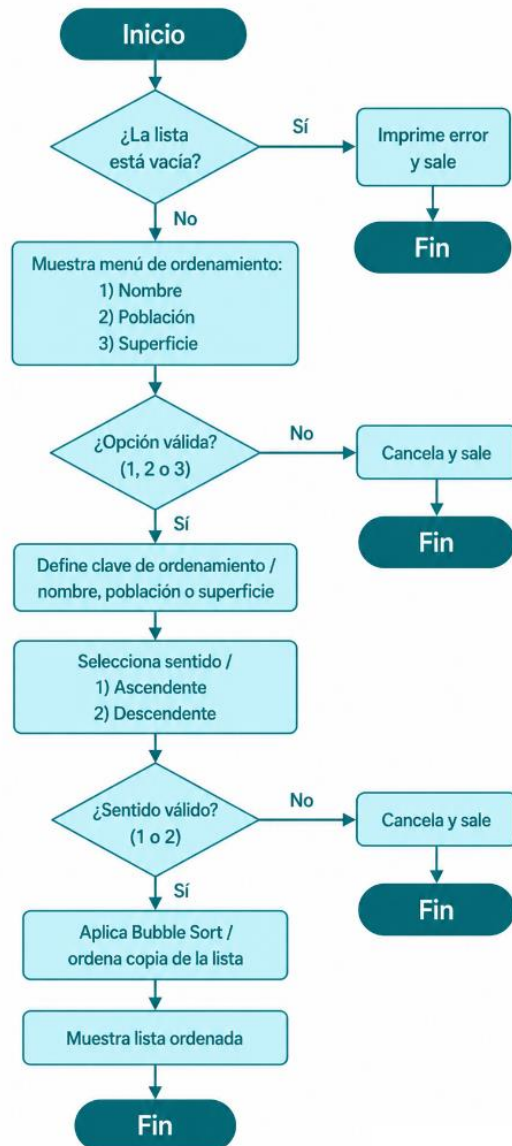
```

Ingrese una opción: 5
FILTRADO DE PAÍSES
1) Por continente
2) Por población
3) Por superficie
4) Cancelar
Seleccione opción: 1
FILTRAR POR CONTINENTE
Ingrese 'x' para cancelar.
Continente: america
Hay 3 países en 'america':

+-----+-----+-----+-----+
| NOMBRE DEL PAÍS | POBLACIÓN | SUPERFICIE KM² | CONTINENTE |
+-----+-----+-----+-----+
| Canada          | 38,000,000 | 9,984,670       | America    |
| Mexico          | 128,000,000 | 1,964,375       | America    |
| Brasil          | 214,000,000 | 8,515,767       | America    |
+-----+-----+-----+-----+
  
```



Ordenar países:



Este diagrama muestra la función `ordenar_paises` que permite visualizar la lista de países en un orden específico sin modificar la lista original.

La vemos en ejecución en la siguiente imagen:

```

Ingrese una opción: 6

ORDENAR PAÍSES
1) Por nombre
2) Por población
3) Por superficie
4) Cancelar

Seleccione una opción: 1

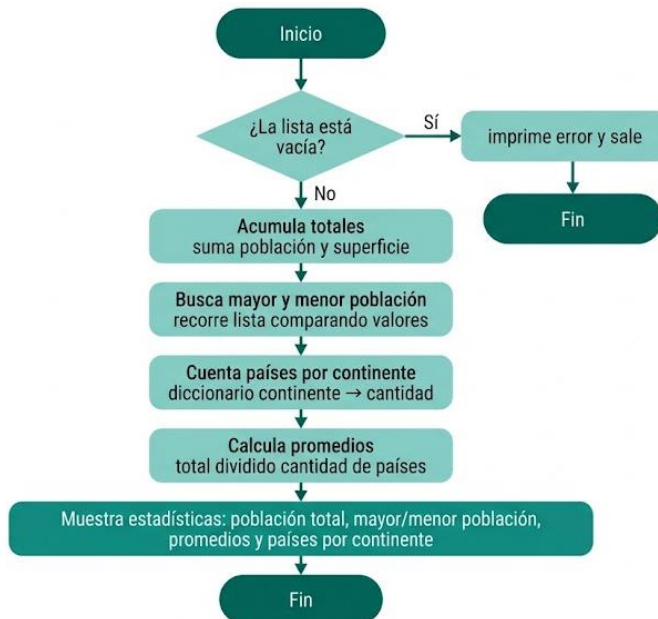
1) Ascendente
2) Descendente

Seleccione sentido: 2

Lista ordenada por NOMBRE:
  
```

NOMBRE DEL PAÍS	POBLACIÓN	SUPERFICIE KM²	CONTINENTE
Tailandia	71,000,000	513,120	Asia
Mexico	128,000,000	1,964,375	America
Japon	125,000,000	377,975	Asia
Italia	59,000,000	301,340	Europa
Espana	47,000,000	505,990	Europa
Egipto	110,000,000	1,002,450	Africa
Corea Del Norte	26,633,691	120,540	Asia
China	1,412,000,000	9,596,960	Asia
Canada	38,000,000	9,984,670	America
Brasil	214,000,000	8,515,767	America
Argentina	46,000,000	2,780,400	America Del sur
Alemania	84,000,000	357,022	Europa

Mostrar estadísticas:



Este diagrama representa la función `mostrar_estadisticas` que recorre la lista de países y calcula una serie de indicadores clave sobre la data set.

En la imagen siguiente lo vemos en ejecución:

Ingrese una opción: 7

ESTADÍSTICAS AVANZADAS

1. Población total acumulada: 2,360,633,691 habitantes.
2. País con MAYOR población: China (1,412,000,000 habitantes).
3. País con MENOR población: Corea Del Norte (26,633,691 habitantes).
4. Promedio de población: 196,719,474.25 habitantes por país.
5. Promedio de superficie: 3,001,717.42 km² por país.
6. Cantidad de países por continente:
 - America Del sur: 1 país(es).
 - America: 3 país(es).
 - Europa: 3 país(es).
 - Africa: 1 país(es).
 - Asia: 4 país(es).

3. Dificultades y conclusiones

Dificultades Técnicas Encontradas y Soluciones

Durante el desarrollo del programa surgieron varios problemas lógicos cuando probábamos el código. A continuación, detallamos los dos obstáculos más grandes que tuvimos y cómo los resolvimos:

1. Problema de las búsquedas rígidas (Mayúsculas y tildes):

El problema: Al comienzo, si en el archivo CSV un país estaba guardado como "España" y el usuario escribía "espana" (en minúscula y sin tilde) en el buscador o para modificarlo, el programa no lo encontraba. Python diferencia estrictamente los caracteres y para el código eran dos palabras totalmente distintas.

La solución: En las funciones de búsqueda y actualización, tomamos tanto el texto que ingresa el usuario como el nombre que está guardado en la lista, y los convertimos temporalmente a minúsculas usando `.lower()`. Además, encadenamos varios comandos `.replace()` para quitar las tildes a mano. De esta forma, el sistema se volvió flexible y encuentra los datos sin importar cómo los tipee el operador.

2. Datos inválidos que rompían las estadísticas o el ordenamiento:

El problema: Cuando programamos la Opción 2 para agregar países, si el usuario ingresaba por error letras en el campo de población o superficie, el programa se cerraba arrojando un error de sistema (`ValueError`). Además, si de alguna forma se guardaba una palabra en esos campos, cuando queríamos utilizar la Opción 6 (ordenar) o la Opción 7 (estadísticas), el

programa colapsaba porque Python no puede sumar ni comparar si un texto es mayor o menor a un número entero.

La solución: La resolvimos blindando las entradas de datos. En lugar de utilizar un input() simple, encerramos la carga de números dentro de un bucle infinito while True con un bloque try-except. Si el usuario introduce letras, el bloque except atrapa el error, muestra un mensaje de advertencia en la consola y vuelve a solicitar el dato de forma obligatoria. El bucle solo se rompe con un break cuando el dato ingresado es un número entero real y mayor a cero, garantizando que la lista de países siempre tenga datos limpios y seguros.

Conclusiones y aprendizajes

La realización de este proyecto integrador nos dejó un gran aprendizaje. Al comienzo de la materia, iniciamos con herramientas sueltas (variables por un lado, ciclos for por otro, funciones o archivos).

Con este trabajo, comprendimos la importancia fundamental de escribir código ordenado y modular (separado en funciones). Si hubiésemos generado todo el programa dentro de un solo bloque gigante, habría sido imposible encontrar los errores de lógica. También nos llevamos como aprendizaje que el desarrollo de software, no se trata solo de que el programa funcione, sino de anticiparse a los errores del usuario y diseñar un sistema robusto que sea capaz de resistirlos.

Este trabajo nos enseñó a desarrollar una base sólida de conceptos y funciones del sistema de programación trabajado: Python.

Bibliografía / Webgrafía

Python Software Foundation. (2026). *csv — CSV File Reading and Writing.* Documentación oficial de Python 3.x. Disponible en: <https://docs.python.org/3/library/csv.html>

Built-in Exceptions (Handling Errors). Documentación oficial de Python 3.x sobre el uso de bloques try-except. Disponible en: <https://docs.python.org/3/library/exceptions.html>

Universidad Tecnológica Nacional (UTN). (2026). *Apuntes de Cátedra: Guía de trabajos prácticos y estructuras de datos dinámicas.* Programación I - Tecnicatura Universitaria en Programación.

CLAUDE.ia. Para generación de diagramas.

Links

- Link al repositorio del proyecto:

https://github.com/Luciana-Valdez/TP_Programacion

- Link del video explicativo:

https://www.youtube.com/watch?v=vRA_szUlnI0